



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Assignment 2

Subject: SECJ2013- Data Structure and Algorithm

Session-Sem: 2024/2025-1

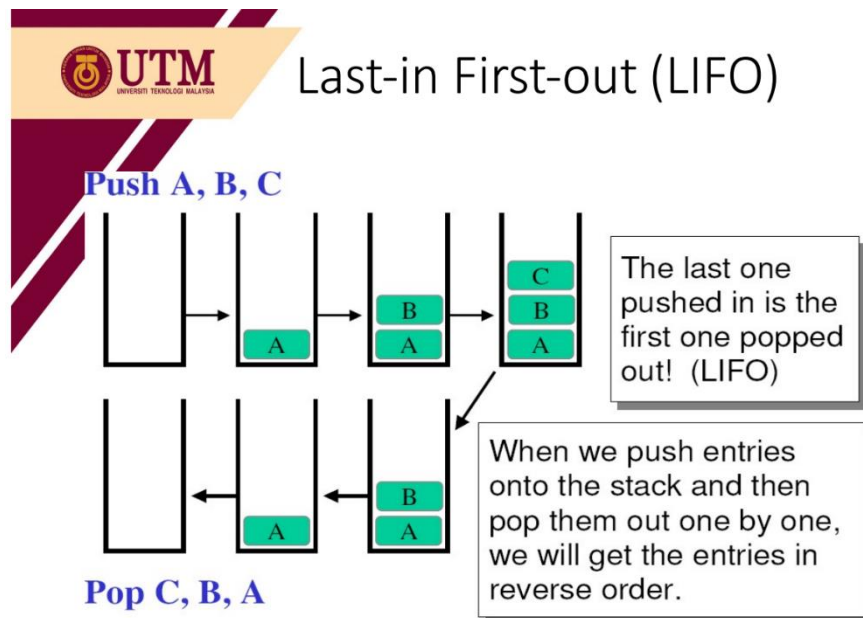
Section: 02

Group: G7

Group Member	Matric Number
Choh Jing Yi	A23CS0296
Joanne Ching Yin Xuan	A23CS0227
Muhammad Afiq Danish bin Mohd Hazni	A23CS0118
Tan Sheau Jun	A22EC8001

Task 1: Understanding the Concept of Stack

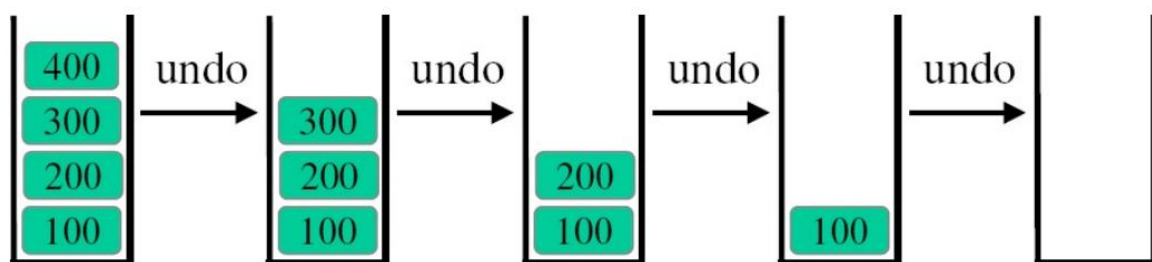
A Stack is a linear data structure that follows a particular order in which the operations are performed. The order may be LIFO (Last In First Out). The LIFO (Last-In-First-Out) principle is a method of organizing and accessing data where the last item added (the most recently added) is the first item to be removed.



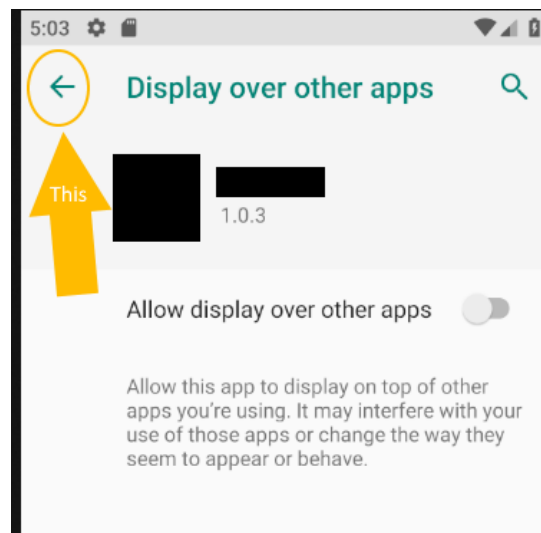
Real world Relevance of LIFO: **Undo** operation.

Scenario: A user input data in row – 100, 200, 300, 400. When user find something wrong, the last one made is undone first when press Undo, return to previous state.

Operation lists:



Real world Relevance of LIFO: **Back operation.**



Scenario: Web browsers keep track of your browsing history in a stack. When a user clicks the "Back" button, the browser goes to the most recent webpage you visited.

Basic operations of a Stack:

1. Push

a. Push (array based)

```
void stack::push(int newItem) {  
    // Check if the stack is full using the isFull() method  
    if (isFull()) {  
        // If the stack is full, print an error message  
        cout << "Sorry, Cannot push item. Stack is now full!" << endl;  
    } else {  
        // Increment the 'top' index to make space for the new item  
        top = top + 1;  
        // Add the new item to the stack at the 'top' position  
        data[top] = newItem;  
    }  
}
```

b. Push (pointer based)

```
void stack::push(char newItem) {  
    // Create a new node using dynamic memory allocation  
    nodeStack *newNode = new nodeStack;  
    // Check if memory allocation for the new node failed  
    if (newNode == NULL) {  
        // If memory allocation failed, print an error message  
        cout << "Cannot allocate memory!" << endl;  
    } else {  
        // Assign the new item's value to the 'data' field of the new node  
        newNode->data = newItem;  
        // Link the new node to the current top of the stack  
        newNode->next = top;  
        // Update the 'top' pointer to point to the new node  
        top = newNode;  
    }  
}
```

3.stackTop

a. stackTop (array-based)

```
int stack::stackTop() {  
    // Check if the stack is empty using the isEmpty() method  
    if (isEmpty()) {  
        // If the stack is empty, print a message indicating this  
        cout << "Sorry, Stack is empty!" << endl;  
    }  
    // Return the value of the element at the top index of the array  
    return data[top];  
}
```

```
}
```

b. stackTop (pointer-based)

```
int stack::stackTop() {  
    // Check if the stack is empty using the isEmpty() method  
    if (isEmpty()) {  
        // If the stack is empty, print a message indicating this  
        cout << "Stack is empty!" << endl;  
        // Return -1 as an indicator that there is no top element  
        return -1;  
    } else {  
        // If the stack is not empty, return the value stored at the top node  
        return top->data;  
    }  
}
```

4. isEmpty()

a. isEmpty(array-based)

```
bool stack::isEmpty() {  
    // Check if the 'top' index is equal to -1  
    // If 'top' is -1, it means the stack has no elements  
    return top == -1;  
}
```

b. isEmpty(pointer-based)

```
bool stack::isEmpty() {  
    // Check if the 'top' index is equal to -1  
    // If 'top' is NULL, it means the stack has no elements  
    return top == NULL;
```

```
}
```

5. IsFull()

a. IsFull(array-based)

```
bool stack::isEmpty() {  
    // Return true if the stack is empty, which is indicated by the 'top' being -1  
    return top == -1;  
}
```

b. IsFull(pointer-based)

```
bool stack::isEmpty() {  
    // Return true if the stack is empty, which is indicated by the 'top' being NULL  
    return top == NULL;  
}
```

Task 2: Application of Stack in a Real-World Scenario

The Stack data structure is applied to the real world for example the implementation of the Undo/Redo functionality of a text editor. For a text editor, efficient implementation of the undo and redo functions is crucial in enhancing user experience since it gives users the ability to backtrack quicker. This functionality relies on the Stack, which revolves around the concept of LIFO. In a text editor, the user's every action-typing, deleting, formatting-a record is stored in an Undo Stack. The 'undo' functionality relies on the most essential feature of a stack: LIFO. It guarantees that, during text processing, the latest action will be the first to get deleted. An example of how applications use the stack data structure involves the usage of two stacks, called the Undo and Redo stacks.

The main stack is the Undo Stack, which keeps a log of the user's activities in reverse order. Whenever any step has occurred, it gets pushed on to the Undo Stack. It saves these actions from most recent to oldest, so the last executed action would always be the first to reach when the 'undo' operation is required. It provides a way for the user to undo his/her last activity. With every user's "Undo" selection, the most recent action is selected from the Undo Stack. For instance, if a user decides to delete a word, upon the user hitting "Undo," it restores it. Further on, this action is put in the Redo Stack to eventually redo it if needed later. This technique keeps up with the users' desired effect of going back yet keeping their ability to make actions active again. The Redo Stack stores list of undone activities that can be reapplied, if necessary. The Redo Stack is initially empty.

While the user Undo, the very last one moves from the Undo Stack into the Redo Stack. This stack ensures each single operation of undo could be rapidly done in such a way that users are allowed to go back when their mind is changed. Hence, when deleting a word and subsequently performing the same action again, the delete action would then be pushed into the Redo Stack. The redo is opposite to an undo. If "Redo" is clicked, then the very last action in the Redo Stack becomes live and performs its act on the text, then goes back in the Undo Stack, all with things in order. In systems that use such mechanism designs, users can always backtrack steps and make changes earlier for a setting that a user wants. If in some application a user enters a word "Algorithm," that gets stored in the Undo Stack. After some editing, for instance deleting a letter "h" results in "Algoritm," such an action itself gets recorded in the same Undo Stack. If

the user clicks "Undo," the previous action of deleting the letter "h" is undone, and the word returns to "Algorithm." The action of removing the letter is added to the Redo Stack.

If the user clicks "Redo," the word will revert to "Algorithm." That's how users can easily switch between actions. Conclusion It is to be concluded that employing two stacks, one for the undo and the other one for redo, provides a powerful and simple method in the handling of changes of text editors. The Undo Stack keeps track of what a user does; the Redo Stack ensures that these undo steps can later be redone if necessary. These two stacks make use of the LIFO concept. Merging the two provides a pretty clear, efficient, and easy way in which users are able to move through editing history. This system contributes a lot to enhancing user experience; on the other hand, the operations are able to be undone in their correct order, making this process quite flexible and user-friendly with a system.

Task 4: Comparative Analysis/Findings

Implementation	Array-Based	Pointer-Based
Memory Usage	Fixed Size	Dynamic size
Ease of Implementation	Simple to implement with indexed access and contiguous memory.	More complex to implement due to pointers and dynamic memory allocation.
Scalability	Limited by predefined size.	Not limited to any size

Array-based implementation typically allocate a fixed-size of memory usage making them predictable. However, the disadvantage of this could be a waste of memory when the amount allocated exceeds the actual usage. In addition, array-based implementation is simpler to implement. This is because array-based implementation relies on simple indexed access and do not require complex memory management. Despite that, resizing of an array requires copying element to a new, larger array. So, it will add the complexity of array-based implementation. Moreover, the scalability of array-based implementation is limited due to predefined size. Hence, array-based implementation is less suitable for scenarios with unpredictable or highly dynamic data sizes.

On the other hand, pointer-based implementation relies on a linked list to dynamically allocate memory for each element in the stack. This approach prevents the waste of memory usage and allow the stack to grow or shrink without being restricted to the predefined size. However, this approach requires managing pointers and dynamic memory allocation which make it more complex compared to array-based implementations. Other than that, pointer-based implementation has higher memory utilization since it typically requires more memory to store pointers for each node. In summary, this approach is ideal for applications that require high scalability and flexibility, especially when the size of the stack is unpredictable and unknown.